Bluetooth Controlled Mini RC Car using ESP32-C3 and DRV8833

Project Report

Submitted by: 1)Anupal Boruah(ECE24-05), 2)Ayush Paul(ECE24-08),
3)Dimpal Ch Dihingia(ECE24-17), 4)Hirok Jyoti Bordoloi(ECE24-20), 5)Nabajit Nath(ECE24-36)

Under the Guidance of: Dr. Grumayum Robert Michael

Department of Electronics And Communucation, Year-2026

Acknowledgement:

We would like to express our deepest appreciation to all those who provided us the possibility to complete this report. A special gratitude we give to our project guide, [Guide Name], whose contribution in stimulating suggestions and encouragement helped us to coordinate our project especially in writing this report.

Furthermore, we would also like to acknowledge with much appreciation the crucial role of the staff of the [Your Department] department, who gave the permission to use all required equipment and the necessary materials to complete the task.

Abstract:

The field of robotics and embedded systems has seen exponential growth, leading to the development of compact, efficient, and wireless autonomous and semi-autonomous vehicles. This project presents the design, development, and implementation of a highly compact Bluetooth-controlled miniature Radio Controlled (RC) car. The core of this robotic vehicle is the ESP32-C3 microcontroller, a modern, low-power RISC-V-based system that offers robust processing capabilities and PWM control. To achieve wireless maneuverability, an HC-05 Bluetooth module is integrated, allowing seamless communication between a smartphone application and the microcontroller.

The mechanical movement is driven by two miniature N12 gear motors, controlled through a DRV8833 dual H-bridge motor driver. Rather than relying on complex mechanical steering linkages or independent steering servos, this vehicle employs a differential steering mechanism. By manipulating the relative rotational speeds and directions of the left and right motors via precise Pulse Width Modulation (PWM) signals, the vehicle can execute sharp turns, pivot in place, and navigate tight spaces. A 3.7V Lithium-Ion battery combined with a TP4056 charging module ensures a reliable, rechargeable power system. This report comprehensively details the hardware architecture, circuit design, software development, and mechanical principles involved in creating this wireless robotic prototype.

Table of Contents:

Introduction:

Miniature robotics serves as an excellent foundational platform for exploring the intersection of embedded software, electronics hardware, and mechanical kinematics. Remote-controlled (RC) cars have long been popular among hobbyists, but integrating modern microcontrollers allows these vehicles to transcend simple toys and become programmable robotic testbeds. This project focuses on building a Bluetooth-controlled mini RC car that prioritizes compactness, efficiency, and modularity.

By utilizing the ESP32-C3 microcontroller, the project steps away from legacy 8-bit systems into 32-bit RISC-V architecture, providing enhanced computational power and superior peripheral management, specifically for motor control. The integration of Bluetooth communication enables user-friendly wireless control via widely available smartphone applications, eliminating the need for custom-built radio transmitters. Furthermore, implementing differential steering significantly simplifies the mechanical chassis design, reducing the overall weight and dimensions of the vehicle while offering superior maneuverability.

Objectives:

The primary objectives of this mini-project are structured to demonstrate practical applications of embedded systems:

- To design and assemble a functional miniature wireless RC car using an ESP32-C3 microcontroller.
- To establish a reliable wireless communication link between a mobile device and the microcontroller using an HC-05 Bluetooth module.
- To interface a DRV8833 motor driver for precise bidirectional control of DC gear motors.
- To implement and demonstrate a differential steering mechanism for vehicle navigation, eliminating the need for steering servos.
- To utilize Pulse Width Modulation (PWM) for variable speed control of the vehicle.
- To design a safe and efficient onboard rechargeable power supply system using a Li-Ion battery and a TP4056 module.

Components Used:

A carefully selected set of hardware components is required to achieve the desired functionality while maintaining a miniature form factor.

- ESP32-C3 Mini Microcontroller Board
- HC-05 Bluetooth Module
- DRV8833 Dual Motor Driver
- N12 Micro Gear Motors (2 units)
- 3.7V Lithium-Ion Battery (Small capacity, e.g., 500mAh - 1000mAh)
- TP4056 Lithium Battery Charging Module
- 1000µF Electrolytic Capacitor
- Micro Wheels compatible with N12 shafts
- Chassis (Custom 3D printed or modified Hot Wheels car body)
- Jumper Wires and Soldering Materials

Description of Components:

This section provides a detailed technical analysis of each component, exploring its specifications and explaining the rationale behind its selection for this specific project.

ESP32-C3 Mini: The ESP32-C3 is a single-core Wi-Fi and Bluetooth 5 (LE) microcontroller SoC, based on the open-source RISC-V architecture. It features a clock speed of up to 160 MHz and provides numerous GPIO pins capable of hardware PWM.

- **Why it is used:** The ESP32-C3 serves as the main brain of the RC car. It was selected because of its exceptionally small footprint (in the "Mini" form factor), its ability to process incoming serial commands from the Bluetooth module, and its advanced LEDC (LED Control) hardware peripheral, which is highly efficient at generating the smooth, high-frequency PWM signals necessary for motor speed control.

HC-05 Bluetooth Module: The HC-05 is a widely used Bluetooth SPP (Serial Port Protocol) module designed for transparent wireless serial connection setup. It operates on the Bluetooth 2.0 standard and communicates with the host microcontroller via a standard UART interface (RX/TX).

- **Why it is used:** It provides the crucial wireless communication link between the user's smartphone and the car. The HC-05 was chosen over the ESP32-C3's internal

Bluetooth Low Energy (BLE) simply because standard Bluetooth Classic (SPP) is vastly easier to interface with generic "Bluetooth RC Controller" apps available on Android, ensuring rapid prototyping and reliable command parsing without complex BLE characteristic configurations.

DRV8833 Motor Driver: The DRV8833 is a dual H-bridge motor driver integrated circuit capable of driving two DC motors or one stepper motor. It supports a wide operating voltage range and can deliver up to 1.5A of continuous current per channel.

- **Why it is used:** Microcontrollers cannot provide enough current directly from their GPIO pins to drive motors; doing so would destroy the chip. The DRV8833 acts as a high-current switch controlled by low-voltage logic. It is specifically selected over older drivers like the L298N because the DRV8833 is highly efficient, generates minimal heat, operates flawlessly at lower battery voltages (down to 2.7V), and possesses a much smaller physical footprint, making it ideal for a miniature chassis.

N12 Gear Motors: N12 motors are ultra-miniature DC motors equipped with an internal all-metal gearbox. They offer high torque at low voltages and have a tiny physical profile.

- **Why it is used:** Standard TT gear motors are too large for a miniature RC car. N12 motors provide the perfect balance of small size, adequate torque to move a small chassis, and low current draw, which preserves battery life.

3.7V Lithium-Ion Battery: A compact, single-cell (1S) Lithium-Ion or Lithium-Polymer battery provides the main energy source for the entire system, typically offering a nominal voltage of 3.7V and a fully charged voltage of 4.2V.

- **Why it is used:** Li-Ion batteries possess a high energy density, meaning they store a lot of energy in a small, lightweight package. This is critical for mobile robotics where weight directly affects speed and battery life.

TP4056 Charging Module: The TP4056 is a complete constant-current/constant-voltage linear charger for single-cell lithium-ion batteries. It usually includes battery protection circuitry (DW01A) to prevent overcharge and over-discharge.

- **Why it is used:** It allows the RC car to be recharged safely via a standard USB cable without needing to remove the battery from the chassis. The included protection circuitry is vital for preventing battery damage if the user runs the car until the battery is completely flat.

1000µF Capacitor: An electrolytic capacitor is a passive component that stores electrical energy in an electric field.

- **Why it is used:** DC motors generate significant electrical noise and experience high inrush currents when starting or stalling. The 1000µF capacitor is placed across the power rails to act as a local energy reservoir. It stabilizes the voltage, absorbs voltage

spikes, reduces motor noise, supports the sudden motor startup current, and prevents the ESP32-C3 from randomly resetting due to voltage drops (brownouts).

System Architecture:

The system architecture of the RC car relies on a decentralized processing flow where the user acts as the command initiator and the ESP32-C3 acts as the central processing hub.

1. **Input Stage:** A smartphone acts as the remote control, capturing user inputs (forward, backward, left, right, speed up, speed down) and translating them into predefined serial character commands.
2. **Communication Stage:** The smartphone transmits these characters via Bluetooth waves to the HC-05 module. The HC-05 converts the wireless signal into standard UART serial data.
3. **Processing Stage:** The ESP32-C3 receives the UART data on its RX pin. The embedded firmware evaluates the character against a programmed control structure.
4. **Output Stage:** Based on the command, the ESP32-C3 generates specific logic and PWM signals.
5. **Actuation Stage:** The DRV8833 motor driver receives the logic signals and amplifies the current from the battery to drive the N12 motors accordingly.

Circuit Connections:

Proper circuit mapping is crucial for the successful operation of the embedded system. Below are the detailed wiring configurations connecting the various modules.

HC-05 Pin	ESP32-C3 Pin / Global	Function
VCC	5V / Battery Positive	Power supply for Bluetooth module
GND	Common GND	Ground reference
TXD	GPIO20 (RX)	Transmits data to ESP32
RXD	GPIO21 (TX)	Receives data from ESP32

DRV8833 Pin	ESP32-C3 Pin / Global	Function
VCC	Battery Positive	Motor power supply
GND	Common GND	Ground reference
IN1	GPIO4	Logic control for Left Motor (Forward)
IN2	GPIO5	Logic control for Left Motor (Reverse)
IN3	GPIO6	Logic control for Right Motor (Forward)
IN4	GPIO7	Logic control for Right Motor (Reverse)
EEP	3.3V (ESP32)	Sleep mode disable (Active High)
ULT	3.3V (ESP32)	Fault pin (Pulled high)
OUT1, OUT2	Left N12 Motor	Power output to left motor
OUT3, OUT4	Right N12 Motor	Power output to right motor

TP4056 Pin	Component	Function
B+	Battery Positive	Charges battery
B-	Battery Negative	Charges battery
OUT+	ESP32 VIN, DRV8833 VCC	Regulated system power
OUT-	Common GND	System ground

Capacitor Connection: The 1000µF electrolytic capacitor is connected in parallel with the main power lines. The positive leg connects to the DRV8833 VCC (OUT+ of TP4056), and the negative leg connects to the Common GND.

Power Supply System:

The power distribution in this miniature RC car must be carefully managed to ensure both the delicate logic components (ESP32, HC-05) and the high-current components (Motors) operate smoothly without interfering with one another.

The core power source is the 3.7V Lithium-Ion battery. The TP4056 module acts as the central power distribution hub. The battery connects to the B+ and B- terminals. All system loads connect to the OUT+ and OUT- terminals. This ensures that the TP4056's onboard DW01A protection IC monitors the discharge limits.

From the OUT+ terminal, the voltage branches into two main paths:

1. **Logic Power:** The voltage is fed into the VIN or 5V pin of the ESP32-C3 mini. The ESP32 board contains an onboard low-dropout (LDO) voltage regulator that steps the raw 3.7V-4.2V battery voltage down to a stable 3.3V for the microcontroller's internal logic.
2. **Motor Power:** The voltage is fed directly into the VCC pin of the DRV8833. Motors are highly tolerant of fluctuating voltages, so they run directly off the unregulated battery voltage to maximize available torque. The 1000µF capacitor is placed close to the DRV8833's VCC and GND pins to act as a buffer against voltage sags caused by the motors.

Working Principle:

The operation of the Bluetooth-controlled RC car follows a precise, sequential flow of signals and electrical power.

When the system is powered on, the ESP32-C3 initializes its GPIO pins for PWM output and opens a hardware serial port to listen for incoming data from the HC-05. The HC-05 module goes into pairing mode, waiting for a connection from the smartphone.

Once the user connects their smartphone application to the HC-05 and presses a directional button (e.g., "Forward"), the application transmits a specific ASCII character (e.g., 'F') over the Bluetooth radio frequency link.

The HC-05 receives this wireless packet and passes the character 'F' to the ESP32-C3 via the UART TX-RX connection. The ESP32 reads the serial buffer. Upon identifying the 'F' command, the microcontroller executes the programmed logic for forward movement. It sets the PWM duty cycles for the forward pins (IN1 and IN3) to the desired speed level while keeping the reverse pins (IN2 and IN4) at logic LOW (0 duty cycle).

These low-voltage PWM signals are fed into the DRV8833 motor driver. The internal H-bridges of the DRV8833 rapidly switch the high-current path from the battery to the motors in synchronization with the PWM signals.

Both N12 motors rotate in the forward direction. The wheels, attached to the motor shafts, grip the surface, and the entire vehicle propels forward. This entire sequence happens in a matter of milliseconds, providing real-time responsiveness to the user's inputs.

Signal Flow Summary: Smartphone HC-05 ESP32-C3 DRV8833 N12 Motors Vehicle Movement

---

Differential Steering Mechanism:

Traditional vehicles utilize Ackermann steering geometry, which requires complex mechanical linkages and an independent servo motor to pivot the front wheels. To maintain a miniature and highly compact design, this RC car abandons mechanical steering in favor of **differential steering** (similar to how a tank operates).

Differential steering relies on the premise that the vehicle can change its trajectory by controlling the speed difference (differential) between the left and right wheels.

- **Forward Movement:** Both left and right motors rotate forward at the same speed.
- **Backward Movement:** Both left and right motors rotate backward at the same speed.
- **Right Turn:** The left motor rotates forward at a high speed, while the right motor rotates forward at a significantly slower speed (or stops entirely). Because the left side is covering more distance than the right side in the same amount of time, the vehicle sweeps into a right-hand turn.
- **Left Turn:** Conversely, during a left turn, the left wheel rotates slower, and the right wheel rotates faster, forcing the vehicle to pivot left.
- **Pivot/Spin in Place:** To achieve a zero-radius turn, the left motor is driven forward while the right motor is driven backward at the same speed (or vice versa).

Advantages of Differential Steering:

- No steering servo required, reducing cost and power consumption.
- Extremely compact structural requirement.
- Simple mechanical design with fewer moving parts to break.
- Exceptional maneuverability, ideal for navigating tight spaces common in miniature RC vehicles.

Bluetooth Communication System:

Wireless data transmission is achieved using the HC-05 Bluetooth SPP module. SPP (Serial Port Profile) is designed to replace a physical serial cable with a wireless Bluetooth connection.

When configuring the system, the ESP32 communicates with the HC-05 using Universal Asynchronous Receiver-Transmitter (UART) protocol. The baud rate is typically set to 9600 bps, which is the default for most HC-05 modules. This speed is more than sufficient for transmitting single-character control commands. The system utilizes an interrupt-driven or polling-based serial read method in the Arduino loop(), ensuring that any incoming command from the smartphone is processed immediately.

PWM Motor Speed Control:

To allow the RC car to move at variable speeds rather than just "full speed" or "stop", the system employs Pulse Width Modulation (PWM). PWM is a technique for getting analog results with digital means. Digital control is used to create a square wave, a signal switched between on and off.

The average voltage fed to the motor is controlled by changing the duty cycle of this square wave. The duty cycle is mathematically defined as:

If the PWM signal is HIGH 50% of the time and LOW 50% of the time, the motor receives roughly 50% of the average voltage, resulting in half speed.

The ESP32-C3 does not use standard analogWrite() like older Arduinos. Instead, it utilizes the advanced **LEDC (LED Control)** peripheral. The LEDC peripheral contains multiple channels capable of generating hardware-based, high-resolution PWM signals without consuming CPU cycles. This ensures highly stable and smooth motor operation.

Software Development:

The firmware for the ESP32-C3 is developed using the Arduino IDE environment, leveraging the vast ecosystem of Arduino libraries combined with the specific ESP32 board support packages. The code is written in Embedded C/C++.

The software architecture is modular, utilizing specific functions for initialization, command parsing, and motor driving.

1. **Hardware Initialization:** The setup() function assigns GPIO pins to specific LEDC channels, sets the PWM frequency (usually around 5000Hz - 20000Hz to push the switching noise above human hearing limits), and initializes the UART serial port.
2. **Continuous Polling:** The loop() function continuously checks the serial buffer. If data is available, it reads the incoming byte.
3. **Command Execution:** A switch-case statement evaluates the byte and calls the corresponding movement function (e.g., moveForward()).

Complete Code Explanation:

Below is the complete, documented C++ code utilized for the project.

Copy#include <Arduino.h>

// Define motor pins corresponding to ESP32-C3 GPIOs

const int IN1 = 4; // Left Motor Forward

const int IN2 = 5; // Left Motor Reverse

const int IN3 = 6; // Right Motor Forward

const int IN4 = 7; // Right Motor Reverse

// Define PWM properties

const int freq = 5000; // 5 kHz frequency

const int resolution = 8; // 8-bit resolution (0-255)

const int IN1_Channel = 0; // LEDC Channel 0

const int IN2_Channel = 1; // LEDC Channel 1

const int IN3_Channel = 2; // LEDC Channel 2

const int IN4_Channel = 3; // LEDC Channel 3

int currentSpeed = 200; // Default PWM duty cycle (0-255)

char command; // Variable to store incoming Bluetooth data

```
void setup() {
  // Initialize Serial communication with HC-05 at 9600 baud rate
  Serial1.begin(9600, SERIAL_8N1, 20, 21); // RX=20, TX=21 for ESP32-C3

  // Configure LEDC PWM channels
  ledcSetup(IN1_Channel, freq, resolution);
  ledcSetup(IN2_Channel, freq, resolution);
  ledcSetup(IN3_Channel, freq, resolution);
  ledcSetup(IN4_Channel, freq, resolution);

  // Attach the GPIO pins to the respective LEDC channels
  ledcAttachPin(IN1, IN1_Channel);
  ledcAttachPin(IN2, IN2_Channel);
  ledcAttachPin(IN3, IN3_Channel);
  ledcAttachPin(IN4, IN4_Channel);

  // Ensure motors are stopped initially
  stopMotors();
}

void loop() {
  // Check if data is available from the Bluetooth module
  if (Serial1.available() > 0) {
    command = Serial1.read();

    // Process the command
    switch(command) {
      case 'F': moveForward(); break;
```

```
      case 'B': moveBackward(); break;
      case 'L': turnLeft(); break;
      case 'R': turnRight(); break;
      case 'S': stopMotors(); break;
      case '1': currentSpeed = 100; break;
      case '2': currentSpeed = 150; break;
      case '3': currentSpeed = 200; break;
      case '4': currentSpeed = 255; break;
    }
  }
}

// Function to move the car forward
void moveForward() {
  ledcWrite(IN1_Channel, currentSpeed); // Left motor forward
  ledcWrite(IN2_Channel, 0);           // Left motor reverse OFF
  ledcWrite(IN3_Channel, currentSpeed); // Right motor forward
  ledcWrite(IN4_Channel, 0);           // Right motor reverse OFF
}

// Function to move the car backward
void moveBackward() {
  ledcWrite(IN1_Channel, 0);
  ledcWrite(IN2_Channel, currentSpeed);
  ledcWrite(IN3_Channel, 0);
  ledcWrite(IN4_Channel, currentSpeed);
}
```

```
// Function to turn left (Differential steering - pivot)
void turnLeft() {
  ledcWrite(IN1_Channel, 0);          // Left motor forward OFF
  ledcWrite(IN2_Channel, currentSpeed); // Left motor reverse ON
  ledcWrite(IN3_Channel, currentSpeed); // Right motor forward ON
  ledcWrite(IN4_Channel, 0);          // Right motor reverse OFF
}

// Function to turn right (Differential steering - pivot)
void turnRight() {
  ledcWrite(IN1_Channel, currentSpeed);
  ledcWrite(IN2_Channel, 0);
  ledcWrite(IN3_Channel, 0);
  ledcWrite(IN4_Channel, currentSpeed);
}

// Function to stop the car
void stopMotors() {
  ledcWrite(IN1_Channel, 0);
  ledcWrite(IN2_Channel, 0);
  ledcWrite(IN3_Channel, 0);
  ledcWrite(IN4_Channel, 0);
}
```

Copy

Code Breakdown:

- **Pin Definitions:** The GPIOs 4, 5, 6, and 7 are mapped to variables for readability.
- **PWM Initialization:** The ledcSetup function configures the frequency and 8-bit resolution (yielding speeds from 0 to 255). ledcAttachPin links the physical GPIOs to the internal timer channels.

- **Bluetooth Serial Communication:** Serial1.begin(9600, SERIAL_8N1, 20, 21) opens a hardware UART port specifically on pins 20 and 21, allowing reliable data capture from the HC-05.
- **Motor Control Logic:** Functions like moveForward() manipulate the duty cycles using ledcWrite. To move forward, the "forward" channels get a positive duty cycle, while the "reverse" channels are set to 0.
- **Turning Logic:** turnLeft() commands the left motor backward and the right motor forward, demonstrating a zero-radius differential turn.

Features:

- **Wireless Bluetooth Control:** Seamless connection via smartphones.
- **PWM Speed Variation:** Dynamic speed adjustments allowing for precise navigation.
- **Differential Steering Capability:** Allows the vehicle to spin in place.
- **Compact Miniature Design:** Small footprint suitable for desktop navigation and small spaces.
- **Rechargeable Battery System:** Integrated TP4056 ensures easy USB charging without disassembly.
- **Lightweight Chassis:** Minimal weight design increases acceleration and battery life.

Advantages:

- **Low Overall Cost:** Built using affordable, readily available hobbyist electronic components.
- **High Portability:** The miniature scale makes it easy to transport and operate anywhere.
- **Educational Value:** Serves as a comprehensive introduction to embedded C programming, circuit design, and basic robotics.
- **Expandable System Architecture:** The ESP32-C3 possesses extra unused GPIO pins, making it easy to add sensors in the future.
- **Low Power Consumption:** The combination of an efficient DRV8833 driver and miniature N12 motors maximizes run time on a small battery.

Limitations:

- **Limited Bluetooth Range:** The standard HC-05 module has a practical range of about 10 meters line-of-sight.
- **Small Battery Backup:** To maintain a small footprint, a small battery is used, limiting continuous operating time before a recharge is needed.
- **No Obstacle Detection:** The current open-loop system requires constant user supervision to avoid collisions.
- **Limited Motor Torque:** N12 motors are excellent for smooth surfaces but struggle on rough terrain or thick carpets.
- **Voltage Dependency:** As the battery discharges, the top speed of the motors slowly decreases because the power system lacks a boost converter.

Applications:

- **Robotics Education:** An ideal project for engineering students learning about microcontroller integration.
- **Embedded Systems Experimentation:** A testbed for learning UART serial communication and hardware PWM.
- **RC Vehicle Prototyping:** A proof-of-concept chassis before scaling up to larger, more expensive robotic builds.
- **Wireless Robotic Systems:** Demonstrates the principles used in larger industrial teleoperated robots.

Future Improvements:

- **Wi-Fi Control Integration:** Replacing the HC-05 with the ESP32-C3's native Wi-Fi to create a web-server-based control interface, greatly extending range.
- **Dedicated Mobile Application:** Developing a custom Android/iOS app tailored specifically for the car's telemetry.
- **Obstacle Avoidance Sensors:** Adding ultrasonic (HC-SR04) or IR sensors to prevent crashes autonomously.
- **Camera Module Integration:** Adding a micro FPV (First Person View) camera system for remote driving out of the line of sight.

- **Closed-Loop Speed Control:** Implementing rotary encoders on the wheels for precise PID speed and position control.

Result:

The fabrication and programming of the Bluetooth-controlled mini RC car were completed successfully. Upon powering the system, the HC-05 module established a stable wireless connection with a designated Android smartphone. The Arduino-compiled firmware correctly parsed serial commands, and the ESP32-C3 generated accurate PWM signals. The DRV8833 motor driver reliably translated these signals to the N12 gear motors. The vehicle demonstrated robust differential steering, successfully moving forward, backward, and executing sharp left and right pivots without any mechanical binding. Furthermore, the TP4056 module successfully recharged the onboard battery, confirming the viability of the power management circuit.

Conclusion:

This project successfully demonstrated the practical integration of modern embedded systems to create a highly functional miniature robotics platform. By utilizing the 32-bit ESP32-C3 microcontroller, the project proved that advanced peripheral management, such as hardware LEDC PWM, significantly improves the smoothness and reliability of motor control compared to legacy microcontrollers. The implementation of wireless communication via the HC-05 Bluetooth module showcased how mobile consumer devices can be seamlessly interfaced with custom hardware.

Furthermore, relying on a differential steering mechanism rather than mechanical steering servo linkages proved to be highly advantageous, allowing the vehicle to achieve a minimal physical footprint while maintaining exceptional agility. The successful interplay between the logic circuits, the DRV8833 motor driver, and the mechanical drive system underscores the core principles of mechatronics. Ultimately, this mini RC car serves as an excellent, scalable foundation for future enhancements in autonomous navigation, IoT integration, and advanced robotic control systems.

References:

- Espressif Systems. *ESP32-C3 Series Datasheet*. Espressif Inc., 2023.
- Texas Instruments. *DRV8833 Dual H-Bridge Motor Driver Datasheet*. TI.com, 2015.
- Arduino Documentation. *Arduino Core for ESP32 - LEDC API Reference*. Arduino.cc.
- Components101. *HC-05 Bluetooth Module Datasheet and Pinout*. Components101.com.
- NanJing Top Power ASIC Corp. *TP4056 1A Standalone Linear Li-Ion Battery Charger Datasheet*.